Informe de Laboratorio 07

Tema: Proyecto Web

Nota

Estudiante	Escuela	Asignatura
Francis Willian Ayamamani Añasco fayamamani@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Programación Web 2 Semestre: I Código: 1702122

Laboratorio	Tema	Duración
07	Proyecto Web	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	Del 20 Julio 2023	Al 21 Julio 2023

1. Tarea

- En sus grupos de trabajo correspondientes. Elabore un servicio web que tenga un CRUD con el uso de este framework.
  - Create - POST
  - Read - GET
  - Update - PUT
  - Delete - DELETE
- Centrarce en el Core business de su aplicacion web. Los mas importante y necesario que este disponible a traves de un servicio web.
- Ejemplos: https://reqbin.com/, https://www.googleapis.com/youtube/v3/playlistItems
- Muestre la funcionalidad consumiendola desde el cliente Rest de su preferencia.
- El metodo GET puede ser directamente consumido por un navegador web:
- Por ejemplo: En esta API se puede obtener la temperatura de Arequipa en un rango de fechas: (La version gratuita tiene un retraso de 7 dias, por tanto solo mostrara la temperatura en Arequipa desde el 01 de Julio hasta el 03 de Julio)
- `https://archive-api.open-meteo.com/v1/archive?latitude=-16.39889&longitude=-71.535&start_date=2023-07-01&end_date=2023-07-10&hourly=temperature_2m&daily=temperature_2m_max,temperature_2m_mintimezone=America%2FNew_York`

2. MARCO CONCEPTUAL

- Editor Vim
  - Vim es un editor de texto muy eficiente para programar en consola que no tiene nada que envidiar a los editores gráficos.
- Instalar Vim en
  - GNU/Linux

```
apt-get install vim
```

  - MS Windows
    - Para descarga en sistemas MS Windows `https://www.vim.org/download.php`
  - MacOS

```
brew install macvim
```

- Git
  - Git es un sistema de control de versiones creado por Linus Torvalds. Es eficiente, confiable.
- Instalar Git
  - GNU/Linux

```
apt-get install git
```

  - MS Windows
    - Para descarga en sistemas MS Windows `https://git-scm.com/download/win`
  - MacOS
    - Para descarga en sistemas MacOS `https://git-scm.com/download/mac`

```
brew install git
```

- python
  - Para un instalación local en sistemas operativos MS Windows utilice: https://www.python.org/downloads/windo
  - Tambien ud puede probar código en herramientas online: `https://www.programiz.com/python-programming/online-compiler/`
  - Python en Debian GNU/Linux:

Listing 1: Debian Linux viene con Python 3 y Python 2 preinstalados.

```
python3 --version
    Python 3.9.2
```

Listing 2: Instalemos pip, una herramienta que instalará y administrará los paquetes de programación que queramos usar en nuestros proyectos de desarrollo.

```
sudo apt-get install -y python3-pip
```

Listing 3: Paquetes y herramientas de desarrollo más para instalar para garantizar que tengamos una configuración sólida para nuestro entorno de programación:

```
sudo apt-get install build-essential libssl-dev libffi-dev
    python3-dev
```

- Configurar un entorno virtual:
  - Los entornos virtuales permiten tener un espacio aislado en los proyectos Python
  - Garantizando que cada proyecto pueda tener su propio conjunto de dependencias que no interrumpirán a otros proyectos.
  - Manejando diferentes versiones de los paquetes. Esto es especialmente importante cuando se trabaja con paquetes de terceros.
  - Puede varios entornos de programación.
  - Cada entorno es un directorio en la que se ubicaran sus scripts.
  - Usaremos el módulo venv , que es parte de la biblioteca estándar de Python.

Listing 4: Entorno de Linux-Instalemos venv escribiendo:

```
sudo apt install -y python3-venv
```

Listing 5: Entorno de Linux-Para crear un ambiente elija en qué directorio.

```
mkdir -p ~/unsa/pw2/labs/lab04/entorno
    cd ~/unsa/pw2/labs/lab04/entorno
```

Listing 6: Entorno de Linux-En el directorio crea un entorno virtual ejecutando el siguiente comando:

```
virtualenv -p python3 .
```

Listing 7: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
source ../bin/activate
```

3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- `https://github.com/rescobedoq/pw2/`
- URL para el laboratorio 05 en el Repositorio GitHub.
- `https://drive.google.com/file/d/16wJ-eMwnGHcARjE6_lBICuXEfWS1QelO/view`

4. Resolucion de la tarea

4.1. Inicio

- Creacion de la carpeta de Trabajo

Listing 8: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
mkdir PWB
```

- Activar el entorno virtual

Listing 9: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
source {{Direccion de tu entorno Virtual}}
```

- Gestionar las versiones instaladas

Listing 10: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip list
```

- Instalar django

Listing 11: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip install Django
```

- Verificas

Listing 12: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip list
 //resultado:
    Package   Version
    ---------- -------
    asgiref   3.5.2
    Django    4.0.5 //Variable
    pip       22.0.4
    setuptools 62.1.0
    sqlparse  0.4.2
    wheel     0.37.1
```

- instalar pillow

Listing 13: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip install pillow
 //resultado:
    Package   Version
    ---------- -------
    asgiref   3.5.2
    Django    4.0.5 //Variable
    pillow    9.5.0 //variable
    pip       22.0.4
    setuptools 62.1.0
    sqlparse  0.4.2
    wheel     0.37.1
```

- Crear el projecto

Listing 14: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
django-admin startproject {{nombre_del_proyecto}}
```

- RESULTADO

```
PWB/
|--- manage.py
|--- PWB/
    |   |--- __init__.py
    |   |--- asgi.py
    |   |--- settings.py
    |   |--- urls.py
```

- Crear el APP para el funcionamiento de una parte del projecto

Listing 15: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
python manage.py startapp {{nombre_app}}
```

- RESULTADO

```
PWB/
|--- manage.py
#nombre de mi APP
|--- Decoraciones/
    |   |--- __init__.py
    |   |--- admin.py
    |   |--- apps.py
    |   |--- models.py
    |   |--- tests.py
    |   |--- views.py
    |   |--- urls.py
|--- PWB/
    |   |--- __init__.py
    |   |--- asgi.py
    |   |--- settings.py
    |   |--- urls.py
```

- Crear el APP para el funcionamiento de la parte del usuario

Listing 16: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
python manage.py startapp {{nombre_app}}
```

- RESULTADO

```
PWB/
|--- manage.py
#nombre de mi APP
|--- Decoraciones/
    |   |--- __init__.py
    |   |--- admin.py
    |   |--- apps.py
    |   |--- models.py
    |   |--- tests.py
    |   |--- views.py
    |   |--- urls.py
```

```
|--- Usuario/
    |   |--- __init__.py
    |   |--- admin.py
    |   |--- apps.py
    |   |--- models.py
    |   |--- tests.py
    |   |--- views.py
    |   |--- urls.py
|--- PWB/
    |   |--- __init__.py
    |   |--- asgi.py
    |   |--- settings.py
    |   |--- urls.py
```

- Cargamos las aplicaciones, que creamos

Listing 17: PWB/settings.py

```
        INSTALLED_APPS = [
            'django.contrib.admin',
            'django.contrib.auth',
            'django.contrib.contenttypes',
            'django.contrib.sessions',
            'django.contrib.messages',
            'django.contrib.staticfiles',
            'Decoraciones',
            'Usuario',
        ]
    )
```

- Establecemos el directorio estatico para el estilo y javascript, en PWB/settings.py

Listing 18: PWB/settings.py

```
    import os
    from pathlib import Path
    #resto del settings.py
    STATIC_URL = 'Decoraciones/static/'
    STATICFILES_DIRS = (
    os.path.join(BASE_DIR,'Decoraciones/static'),
    )
```

- Luego determinamos la ubicacion de los templates

Listing 19: PWB/settings.py

```
    TEMPLATES = [
    #resto de codigo
        'DIRS':
            ['/home/francis/Documentos/pweb/pw2-lab-d-23a/lab5/PWEB/Decoraciones/Templates'],
    #resto de codigo
```

- Luego creamos el directorio media/ para cargar las imagenes

Listing 20: PWB/settings.py

```
import os
from pathlib import Path
    #resto del settings.py
MEDIA_URL = 'Decoraciones/media/'
MEDIA_ROOT = os.path.join(BASE_DIR,'Decoraciones/media')
```

4.2. Explicacion del Codigo

Listing 21: Decoraciones/models.py

```
from django.db import models

class Tipo(models.Model):
    Tip = models.CharField(max_length=50, default=None, null=True)

    def __str__(self):
        return self.Tip

class Producto(models.Model):
    nombre = models.CharField(max_length=25)
    descripcion = models.TextField()
    precio = models.CharField(max_length=25)

    class Meta:
        abstract = True

class catalogo(Producto):
    imagen = models.ImageField(null=True, blank=True, upload_to="img/")
    tipo = models.ForeignKey(Tipo, on_delete=models.CASCADE, null=True)

    def __str__(self):
        return self.nombre

class Carrito(Producto):
    adquirido = models.BooleanField()
    rutaI = models.CharField(null=True, max_length=50000)

class Pedido(models.Model):
    NombreC = models.CharField(null=True, max_length=50)
    ApellidosC = models.CharField(null=True, max_length=50)
    telefono = models.IntegerField(null=True)
    Correo = models.EmailField(max_length=254, default='')
    direccion = models.CharField(max_length=50)
    fecha = models.DateField()
    Carrito = models.ForeignKey(Carrito, on_delete=models.CASCADE, null=True)
```

- **Tipo**: Es un modelo que representa el tipo de producto. Tiene un campo Tip que es un CharField de máximo 50 caracteres de la cadena de texto. La función**str** devuelve la vista del admin, este caso sería el valor del campo Tip.
- **Producto**: Es una clase abstracta que representa un producto genérico. Tiene tres campos: nombre, descripcion y precio. Estos campos son un CharField, un TextField y otro CharField

respectivamente. Al declarar **abstract = True** en la clase Meta, se indica que este modelo es abstracto.

- **catalogo**: Este hereda de Producto y representa un producto en particular.Integra un campo imagen que es un ImageField para almacenar la imagen del producto, y un campo tipo que es una clave externa (**ForeignKey**) que se relaciona con el modelo Tipo.
- **Carrito**: Es un modelo que también hereda de Producto y representa un producto agregado al carrito de compras. Además de los campos heredados de Producto, tiene un campo adquirido que es un BooleanField para indicar si el producto ha sido adquirido, y un campo rutaI que es un CharField para almacenar la ruta de la imagen del producto.
- **Pedido**: Es un modelo que representa un pedido realizado por un cliente. Tiene varios campos como NombreC, ApellidosC, telefono, Correo, direccion, fecha y Carrito. Estos campos representan información relacionada con el pedido, como el nombre del cliente, apellidos, teléfono, correo, dirección, fecha del pedido, y un campo Carrito que es una clave externa (ForeignKey) que se relaciona con el modelo Carrito.

Listing 22: Decoraciones/models.py

```
from django.contrib import admin
from Decoraciones.models import Pedido,catalogo,Carrito,Tipo
# Register your models here.
admin.site.register(Tipo)
admin.site.register(Pedido)
admin.site.register(catalogo)
admin.site.register(Carrito)
```

- **La línea from django.contrib import admin** importa el módulo admin de Django, que proporciona las funcionalidades relacionadas con el panel de administración.
- Luego se importan los modelos Pedido, catalogo, Carrito y Tipo desde el módulo models de la aplicación Decoraciones.
- La función **admin.site.register(Modelo)** se utiliza para registrar cada modelo en el panel de administración. Aquí, se registran los modelos Tipo, Pedido, catalogo y Carrito.

Listing 23: Decoraciones/views.py

```
from django.shortcuts import render
from Decoraciones.models import catalogo
# Lista de elementos
list_producto=catalogo.objects.all()
#Pagina de Inicio
def Home(request):
    return render(request,"home.html",{"producs":list_producto})
def Bebidas(request):
    return render(request,"Bbs.html",{"producs":list_producto})
def Bocaditos(request):
    return render(request,"Bts.html",{"producs":list_producto})
def Equipos(request):
    return render(request,"Eps.html",{"producs":list_producto})
def Decoraciones(request):
    return render(request,"Dec.html",{"producs":list_producto})
```

- **from Decoraciones.models import catalogo**.Importa el modelo catalogo desde el archivo models.py del directorio Decoraciones. Esto permite acceder a los objetos de la clase catalogo definida en ese modelo.
- $list_producto$: $catalogo.objects.all()$, obtiene todos los objetos de la clase catalogo almacenados y los asigna a la variable $list_producto$. Esto devuelve una lista de todos los productos.
- **def Home(request)**, es una función que recibe un objeto request como parámetro. Esta función se utiliza para manejar el template **home.html**.
- **request,"home.html",producs:**$list_producto$, renderiza la plantilla home.html y la devuelve como respuesta HTTP. Además, se pasa un diccionario como contexto a la plantilla, donde la clave "producs.[es] $list_producto$. Esto permite acceder a la lista de productos en la plantilla para mostrarlos.
- Las funciones Bebidas, Bocaditos, Equipos y Decoraciones siguen la misma estructura que la función Home. Cada una maneja una solicitud específica para mostrar una página correspondiente a cada una de ellas.

Listing 24: Decoraciones/urls.py

```
from django.urls import path
from Decoraciones import views
urlpatterns = [
    path('', views.Home, name="Inicio"),
    path('Bebidas/', views.Bebidas, name="Bebidas"),
    path('Bocaditos/', views.Bocaditos, name="Bocaditos"),
    path(' Equipos/', views.Equipos, name="Equipos"),
    path('Decoraciones/', views.Decoraciones, name="Decoraciones"),
]
```

- **urlpatterns = [...]**, es una lista que contiene todas las rutas URL de la aplicación y las funciones de vista asociadas a cada una.
- **path(", views.Home, name=Ïnicio")**, define una ruta URL vacía (") que corresponde a la página de inicio de la aplicación. Asocia esta ruta con la función de vista Home del módulo views. Además, le asigna el nombre Ïnicio.[a] esta ruta, lo que permite referenciarla fácilmente en otros lugares del código.
- Y de igual manera path('Bebidas/','Bocaditos/','Equipos/','Decoraciones/').Definen una ruta URL que corresponde a una parte de la página de decoraciones de la aplicación. Asocia esta ruta con la funcion correspondiente y le asigna el nombre "Decoraciones"..

Listing 25: PWB/urls.py

```
from django.contrib import admin
from django.urls import path,include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('Decoraciones.urls')),
    path('Usuario/', include('Usuario.urls')),
    path('Usuario/', include('django.contrib.auth.urls')),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

- from django.urls import path, include: Importa los módulos path e include de Django. path se utiliza para definir las rutas URL del sitio web, mientras que include se utiliza para incluir URLs de otras aplicaciones dentro del proyecto.
- from django.conf import settings: Importa el módulo settings de Django, que contiene la configuración del proyecto.
- from django.conf.urls.static import static: Importa la función static del módulo urls de Django, que se utiliza para servir archivos estáticos, como imágenes, desde una URL específica.
- static(settings.$MEDIA_URL$, $document_root$=settings.$MEDIA_ROOT$): Agrega la configuración para servir archivos de medios (como imágenes subidas por los usuarios) desde la ruta URL especificada en $MEDIA_URL$, utilizando la ubicación definida en $MEDIA_ROOT$.

Listing 26: Usuario/forms.py

```
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
from django import forms
from Decoraciones.models import Pedido

class Registro(UserCreationForm):
    email = forms.EmailField(max_length=50)
    last_name= forms.CharField(max_length=120)
    first_name= forms.CharField(max_length=120)

    class Meta:
        model = User
        fields = ('username','first_name','last_name','email','password1','password2')

class Pedidos(forms.ModelForm):

    class Meta:
        model = Pedido
        fields = ('NombreC','ApellidosC', 'telefono','Correo','direccion','fecha')

def __init__(self, *args, **kwargs):
super().__init__(*args, **kwargs)
self.fields['fecha'].label = 'Fecha (A-M-D)'
```

- Registro:
  - Hereda de UserCreationForm, que es un formulario proporcionado por Django para el registro de usuarios.
  - Agrega tres campos adicionales al formulario: email, $last_name$ y $first_name$. Estos campos se representan como campos de texto (forms.CharField) y tienen una longitud máxima definida.
  - La clase Meta especifica que el modelo asociado al formulario es User (del módulo django.contrib.auth.models), y enumera los campos que se deben mostrar en el formulario, incluyendo los campos heredados de UserCreationForm.
- Pedidos:
  - Hereda de forms.ModelForm, que es una clase base para crear formularios basados en modelos en Django.

- La clase Meta especifica que el modelo asociado al formulario es Pedido (del archivo models.py en la aplicación Decoraciones), y enumera los campos que se deben mostrar en el formulario.
- El método init es un método especial que se llama al instanciar el formulario. En este caso, se utiliza para personalizar la etiqueta del campo fecha y cambiarla a "Fecha (A-M-D)"

Listing 27: Usuario/urls.py

```
from django.urls import path
from . import views
urlpatterns = [
    path('Ingreso/', views.login_U, name="Login"),
    path('salida/', views.logout_U, name="Logout"),
    path('registro/', views.regist_U, name="Registro"),
    path('Contrato/', views.Proforma, name="contrato"),
    path('Agregar/', views.agregar, name="carrito"),
    path('lista/', views.lista, name="bolsa"),
    path('Borrar/<int:numero>/', views.borrar, name='borrar'),
]
```

- Este urls, es simetrico, respeto a los path y las importaciones de Decoraciones/urls.py.
- **path('Borrar/¡int:numero¿/'**. Define una URL con el patrón "Borrar/int:numero/"donde ¡int:numero¿es un parámetro numérico que se pasa a la vista borrar en el módulo views.py. También se le asigna el nombre "borrar".

Listing 28: Usuario/views.py

```
from django.shortcuts import render,redirect
from django.contrib.auth import authenticate,login, logout
from django.contrib import messages
from django.db.models import Sum
from Usuario.forms import Registro,Pedidos
from Decoraciones.models import Carrito
# Create your views here.

#Lista de productos.
List_prod=Carrito.objects.all()

def login_U(request):
    if request.method == "POST" :
        nombre=request.POST['username']
        contrasena=request.POST['password']
        usuario=authenticate(request,username=nombre,password=contrasena)
        if usuario is not None:
            login(request, usuario)
            return redirect('Inicio')
        else:
            messages.success(request, "Ocurrio -- Erro -- 403")
            return redirect('Login')
    else:
        return render(request, "login.html",{})

def logout_U(request):
```

```
        logout(request)
        #falta completar
        #Carrito.objects.all().delete()
        return redirect('Login')
    def regist_U(request):
        if request.method == "POST":
            form = Registro(request.POST)
            if form.is_valid():
                form.save()
                nombreN = form.cleaned_data['username']
                passN = form.cleaned_data['password1']
                usuarioA = authenticate(username=nombreN,password = passN)
                login(request,usuarioA)
                return redirect('Inicio')
        else:
            form=Registro()
        return render(request, 'registro.html',{"formulario":form})
    def Proforma(request):
        if request.method == "POST":
            contrato = Pedidos(request.POST)
            if contrato.is_valid():
                contrato.save()
                contrato.carrito = Carrito.objects.all()
        else:
            contrato=Pedidos()
        return render(request, 'Proforma.html',{"Ped":contrato,"productos":List_prod})
    def agregar(request):
        if request.method == "POST":
            nombreC = request.POST.get('nombre')
            descripcionC = request.POST.get('descripcion')
            precioC = request.POST.get('precio')
            urlC= request.POST.get('imagen_url')
            Articulo= Carrito(nombre=nombreC, descripcion=descripcionC, precio=precioC,
                adquirido=True, rutaI=urlC)
            Articulo.save()
            return redirect ("Inicio")
    def borrar(request, numero):
        articulo = Carrito.objects.get(id=numero)
        articulo.delete()
        return redirect("bolsa")
    def lista(request):
        listaArticulo=Carrito.objects.all()
        total = Carrito.objects.aggregate(Sum('precio'))['precio__sum'] or 0
        return render(request, 'lista.html', {"productos":listaArticulo,"total":total})
```

- $login_U$(request): Verifica las credenciales de inicio de sesión proporcionadas por el usuario. Si las credenciales son válidas, el usuario se autentica y se redirige a la página de inicio (Inicio). Si las credenciales no son válidas, se muestra un mensaje de error y se redirige de nuevo a la página de inicio de sesión (Login).
- $logout_U$(request): Cierra la sesión del usuario y redirige a la página de inicio de sesión (Login).
- $regist_U$(request): Maneja el proceso de registro de nuevos usuarios. Si se envía un formulario válido, se crea un nuevo usuario, se autentica y se redirige a la página de inicio (Inicio). Si el formulario no es válido, se muestra el formulario de registro nuevamente.

- Proforma(request): Maneja la creación de un nuevo pedido. Si se envía un formulario válido, se guarda el pedido y se muestra la página de proforma. Si el formulario no es válido, se muestra el formulario de pedido.
- agregar(request): Agrega un artículo al carrito de compras. Obtiene los datos del artículo del formulario enviado y crea un nuevo objeto Carrito en la base de datos. Luego, redirige a la página de inicio (Inicio).
- borrar(request, numero): Elimina un artículo del carrito de compras. Obtiene el artículo correspondiente al número proporcionado y lo elimina de la base de datos. Luego, redirige a la página del carrito de compras (bolsa).
- lista(request): Muestra la lista de artículos en el carrito de compras. Obtiene todos los objetos Carrito de la base de datos y calcula el total de precios de los artículos. Luego, renderiza la página de la lista de artículos (lista.html) con los datos correspondientes.

4.3. Templates

Listing 29: Decoraciones/Templates/inicio.html

```
        <!DOCTYPE html>
    <html lang="en">
    <head>
        {% load static %}
        <link rel="preconnect" href="https://fonts.googleapis.com">
        <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
        <link href="https://fonts.googleapis.com/css2?family=Gasoek+One&display=swap"
            rel="stylesheet">
        <link href="https://fonts.googleapis.com/css2?family=PT+Sans&display=swap"
            rel="stylesheet">
        <link href="https://fonts.googleapis.com/css2?family=Dosis:wght@300&display=swap"
            rel="stylesheet">
        <link rel="stylesheet"
            href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.3/css/all.min.css"
            integrity="sha512-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
            crossorigin="anonymous" />
        <link rel="stylesheet" href="{% static 'Decoracions/style/style.css' %}">
        <script src="https://kit.fontawesome.com/eb496ab1a0.js"
            crossorigin="anonymous"></script>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title> ALILEO{% block title %} {% endblock %}</title>
    </head>
    <body>
            <header>
                <h2>
                    {% block sub %}{% endblock%}
                </h2>
            </header>
            <nav>
                <ul id="barra">
                    <li><a href="/">Home</a></li>
                    <li><a href="/Bebidas/">Bebidas</a></li>
                    <li><a href="/Bocaditos/">Bocaditos</a></li>
                    <li><a href="/Equipos/">Equipos</a></li>
                    <li><a href="/Decoraciones/">Decoraciones</a></li>
```

```
                {% if not user.is_authenticated %}
                <li><a href="/Usuario/Ingreso/">Acceder</a></li>
                {% else %}
                    {% if user.is_superuser %}
                        <li><a href="/admin/"><i class="fas
                            fa-user"></i>-ADMINISTRAR</a></li>
                        <li><a href="/admin/"><i class="fas
                            fa-shopping-cart"></i>PEDIDOS</a></li>
                        <li><a href="/Usuario/salida/"><i class="fas
                            fa-power-off"></i></a></li>
                    {% else %}
                        <li class="nav-item"><a class="nav-link"
                            href="/Usuario/lista/"><i class="fas fa-user"></i>U</a></li>
                        <li><a href="/Usuario/Contrato/">Contrato</a></li>
                        <li><a href="/Usuario/salida/"><i class="fas
                            fa-power-off"></i></a></li>
                    {% endif %}
                {% endif %}
            </ul>
        </nav>
        <main>
        <form action="/Usuario/Agregar/" method="POST">
               {% csrf_token %}
            <input type="hidden" name="imagen_url" id="imagen_url">
            <input type="hidden" name="nombre" id="nombre_input">
            <input type="hidden" name="descripcion" id="descripcion_input">
            <input type="hidden" name="precio" id="precio_input">
            <section class="zoom" id="zoom">
                <img src="" id="zImagen">
                <div class="at" id="at">
                    <p id="nombre"></p>
                    <p id="descripcion"></p>
                    <p id="precio"></p>
                    {% if not user.is_superuser and user.is_authenticated %}
                    <button type="submit"><i class="fas fa-shopping-cart"
                        style="background-color:rgb(255, 115, 0);"></i></button>
                    {% endif %}
                </div>
                <span onclick="cerrar()">X</span>
            </section>
        </form>
            <section id="galeria">
                {% block content %}
                {% endblock%}
            </section>
        </main>
        <footer class="pie-pagina">
            <div class="grupo-1">
                <div class="box">
                    <figure>
                        <a href="#">
                            <img id="logo" src="{% static 'Decoracions/style/logo.png'
                                %}" alt="Logo de SLee Dw">
                        </a>
                    </figure>
                </div>
```

```
                    <div class="box">
                        <h2>SOBRE NOSOTROS</h2>
                        <p>NUestro servicio se ...</p>
                        <p>Politica ...</p>
                    </div>
                    <div class="box">
                        <h2>SIGUENOS</h2>
                        <div class="red-social">
                            <a href="#" class="fa fa-facebook"></a>
                            <a href="#" class="fa fa-instagram"></a>
                            <a href="#" class="fa fa-twitter"></a>
                            <a href="#" class="fa fa-youtube"></a>
                        </div>
                    </div>
                </div>
            </footer>
            <script src="{% static 'Decoracions/js/index.js' %}"></script>
    </body>
    </html>
```

- **load static**, carga los archivos estáticos (como CSS y JavaScript) en la plantilla.
- **block title, endblock en title**, define el título de la página. Puede ser modificado por bloques en las plantillas que hereden de esta página base.
- block sub - endblock en la etiqueta h2. Define un bloque que puede ser reemplazado por las plantillas que hereden de esta página base. Se utiliza para mostrar el título o subtitulos en el encabezado.
- ¡nav¿: Inicia la sección de navegación. Id:barra, inicia una lista desordenada para los elementos de navegación:
  - Home, es un enlace a la página de inicio.
  - Bebidas, es un enlace a la página de bebidas.
  - Bocaditos es un enlace a la página de bocaditos.
  - Equipos es un enlace a la página de equipos.
  - Decoraciones es un enlace a la página de decoraciones.
- if not $user.is_authenticated$: Verifica si el usuario no ha iniciado sesión.
- **href: /Usuario/Ingreso/-¿Acceder**: Crea un enlace para que el usuario acceda a la página de inicio de sesión.
- if $user.is_superuser$: Verifica si el usuario es un superusuario (administrador).
- -ADMINISTRAR: Crea un enlace al panel de administración para los superusuarios.
- PEDIDOS: Crea un enlace a la página de pedidos para los superusuarios.
- href: /Usuario/salida/, **i class="fas fa-power-off"**: Crea un enlace para que el usuario cierre sesión.
- **i class="fas fa-userU**: Crea un enlace a la lista de artículos del usuario.
- Contrato: Crea un enlace a la página para crear contratos.

- $csrf_token$: Inserta un token de seguridad CSRF en el formulario.
- Los elementos input ocultos se utilizan para capturar información sobre un artículo que se agregará al carrito de compras.
- section class=”zoom”: Inicia una sección para mostrar detalles ampliados de un artículo cuando se hace clic en él.
- i class=”fas fa-shopping-cart”: Creado en un botón es un icono para reconocer el envío que agrega el artículo al carrito de compras.
- span X: Crea un botón para cerrar la vista ampliada del artículo.
- block content - endblock: Define un bloque que puede ser reemplazado por las plantillas que hereden de esta página base. Se utiliza para mostrar el contenido específico de cada página.
- footer: Inicia la sección del pie de página.
- div class=”box”: Inicia un div para mostrar el logo.
- static ’Decoracions/style/logo.png’: Carga el logo de la página, ubicada en static/Decoracions/style/logo.png.
- static ’Decoracions/js/index.js’: Carga el archivo.js, ubicado en static/Decoracions/js/index.js

Listing 30: Decoraciones/Templates/home.html

```
{% extends 'inicio.html' %}
{% load static %}
{% block title%}
HOME
{% endblock %}
{% block sub %}
DECORACIONES </br> BUFFET
{% endblock%}
{% block content %}
{% for producto in producs %}
<img src="{{producto.imagen.url}}" alt="{{producto.nombre}}"
    onclick="Open('{{producto.imagen.url}}','{{producto.nombre}}','{{producto.descripcion|linebrea
{% endfor %}
{% endblock %}
```

- extends ’inicio.html’: Esta línea indica que la plantilla actual (lista.html) hereda del archivo de plantilla base llamado inicio.html. Esto significa que la plantilla actual extenderá y reemplazará los bloques definidos en el archivo base.
- HOME: El contenido ”HOME”se encuentra dentro del bloque ”title”. Este será el título de la página.
- DECORACIONES ¡/br¿BUFFET: El contenido ”DECORACIONES ¡/br¿BUFFET”se encuentra dentro del bloque ”sub”. Este será el texto adicional en la página.
- for producto in producs: Este es un bucle for de Django que itera sobre una variable llamada producs. Cada iteración del bucle se refiere a un objeto producto.
- img:Es una etiqueta de imagen HTML (¡img¿) que representa un producto. Los atributos src, alt y onclick se rellenan con los valores de las propiedades del objeto producto.

Listing 31: Decoraciones/Templates/Eps.html

```
{% extends 'inicio.html' %}
{% block title%}
EQUIPOS
{% endblock %}
{% block sub %}
EQUIPOS
{% endblock%}
{% block content %}
{% for producto in producs %}
{% if producto.tipo.Tip == "Equipo" %}
<img src="{{producto.imagen.url}}" alt="{{producto.nombre}}"
    onclick="Open('{{producto.imagen.url}}','{{producto.nombre}}','{{producto.descripcion|linebrea
{% endif %}
{% endfor %}
{% endblock %}
```

- if producto.tipo.Tip == .ᴱᑫuipo": Esta línea verifica si el valor de la propiedad Tip del objeto producto.tipo es igual a .ᴱᑫuipo". Es una condición que filtra los productos y solo muestra aquellos que cumplen con esta condición.

Listing 32: Decoraciones/Templates/Dec.html

```
{% extends 'inicio.html' %}
{% block title%}
DECORACION
{% endblock %}
{% block sub %}
DECORACIONES
{% endblock%}
{% block content %}
{% for producto in producs %}
{% if producto.tipo.Tip == "Decoraciones" %}
<img src="{{producto.imagen.url}}" alt="{{producto.nombre}}"
    onclick="Open('{{producto.imagen.url}}','{{producto.nombre}}','{{producto.descripcion|linebrea
{% endif %}
{% endfor %}
{% endblock %}
```

Listing 33: Decoraciones/Templates/Bts.html

```
{% extends 'inicio.html' %}
{% block title%}
BOCADITOS
{% endblock %}
{% block sub %}
BOCADITOS
{% endblock%}
{% block content %}
{% for producto in producs %}
{% if producto.tipo.Tip == "Bocadito" %}
<img src="{{producto.imagen.url}}" alt="{{producto.nombre}}"
    onclick="Open('{{producto.imagen.url}}','{{producto.nombre}}','{{producto.descripcion|linebrea
{% endif %}
```

```
{% endfor %}
{% endblock %}
```

Listing 34: Decoraciones/Templates/Bbs.html

```
{% extends 'inicio.html' %}
{% block title%}
BEBIDAS
{% endblock %}
{% block sub %}
BEBIDAS
{% endblock%}
{% block content %}
{% for producto in producs %}
{% if producto.tipo.Tip == "Bebida" %}
<img src="{{producto.imagen.url}}" alt="{{producto.nombre}}"
    onclick="Open('{{producto.imagen.url}}','{{producto.nombre}}','{{producto.descripcion|linebrea
{% endif %}
{% endfor %}
{% endblock %}
```

- de la misma manera que Eps.html; Dec.html,Bts.html y Bbs.html,estos solo acepta un solo tipo de elemento

Listing 35: Decoraciones/Templates/lista.html

```
{% extends 'inicio.html' %}
{% load static %}
{% block title%}
CONOCER
{% endblock %}
{% block sub %}
PROFORMA </br>
{% endblock%}
{% block content %}
<div>
    <h3 style=" background-color: rgb(255, 115, 0);text-align:
        center;">BIENVENIDO {{user.username}}</h3>
    <h3 style="text-align: center; color:white;background-color:rgba(255, 204,
        162, 0.699);">T-U L-I-S-T-A</h3>
    <h4 style="background-color: rgb(255, 115, 0);text-align: center;
        color:white;">TOTAL: S/.{{total}}</h4>
    <div id="lista">
        {% for producto in productos %}
            <div id="articulos">
                <p id="nombre">{{producto.nombre}}</p>
                <img src= {{producto.rutaI}} alt="{{producto.nombre}}">
                <p id="precio">{{ producto.precio }}<a
                    href="/Usuario/Borrar/{{producto.id}}/"><i class="fas
                    fa-trash-alt"></i></a></p>
            </div>
        {% endfor %}
    </div>
</div>
{% endblock %}
```

- BIENVENIDO user.username: Se muestra un encabezado de nivel 3 (¡h3¿) que muestra un saludo de bienvenida al usuario actualmente autenticado. El nombre de usuario se obtiene a través de la variable user.username.
- TOTAL: S/.total: Se muestra un encabezado de nivel 4 (¡h4¿) que muestra el texto "TOTAL: S/."seguido del valor de la variable total. El estilo CSS se aplica para cambiar el color de fondo y el color del texto.
- lista: Se abre un contenedor de división (¡div¿) con un atributo de identificación (id) establecido como "lista". Este contenedor se utiliza para mostrar una lista de productos.

Listing 36: Decoraciones/Templates/Proforma.html

```
{% extends 'inicio.html' %}
{% load static %}
{% block title%}
CONOCER
{% endblock %}
{% block sub %}
CONTRATO </br> PROFORMA
{% endblock%}
{% block content %}
<div id="contraste">
<form method="POST" >
    {% csrf_token %}
    <div id="caracteristicas">
    {{Ped}}
</div>
    <input type="submit" value="Contratar">
</form>
</div>
{% endblock %}
```

- Ped: Esta línea muestra el contenido de la variable Ped, que se espera que contenga las características del contrato o proforma.

Listing 37: Decoraciones/Templates/registro.html

```
<link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
    rel="stylesheet"
    integrity="sha384-9ndCyUaIbzAi2FUVXJi0CjmCapSmO7SnpJef0486qhLnuZ2cdeRhO02iuK6FUUVM"
    crossorigin="anonymous">
<body>
    <h2>Registrase ZZAZ</h2>
    {% if formulario.errors %}
    <div class="alert alert-warning alert-dismissible fade show" role="alert">
        Se genero un Error...
    <button type="button" class="btn-close" data-bs-dismiss="alert"
        aria-label="Close"></button>
    </div>
    {% endif %}
    <form method="POST">
        {% csrf_token %}
        {{formulario.as_p}}
```

```
        <input type="submit" value="Registrarse">
    </form>
</body>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-geWF76RCwLtnZ8qwWowPQNguL3RmwHVBC9FhGdlKrxdiJJigb/j/68SIy3Te4Bkz"
    crossorigin="anonymous"></script>
```

- $formulario.as_p$: Esta línea muestra los campos del formulario (formulario) utilizando el formato as_p de Django. Esto generará los campos del formulario en formato de párrafos
- script: Esta línea agrega un enlace al archivo JavaScript de Bootstrap. El enlace especifica la ubicación del archivo JavaScript y contiene atributos de integridad y crossorigin para garantizar que se cargue de forma segura.
- link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css": Esta línea agrega un enlace al archivo CSS de Bootstrap. El enlace especifica la ubicación del archivo CSS y contiene atributos de integridad y crossorigin para garantizar que se cargue de forma segura.
- Activar el entorno virtual

Listing 38: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
source {{Direccion de tu entorno Virtual}}
```

- Gestionar las versiones instaladas

Listing 39: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip list
```

- Instalar django

Listing 40: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip install Django
```

- Verificas

Listing 41: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip list
 //resultado:
    Package   Version
    ---------- -------
    asgiref   3.5.2
    Django    4.0.5 //Variable
    pip       22.0.4
    setuptools 62.1.0
    sqlparse  0.4.2
    wheel     0.37.1
```

- instalar pillow

Listing 42: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip install pillow
 //resultado:
    Package   Version
    ---------- -------
    asgiref   3.5.2
    Django    4.0.5 //Variable
    pillow    9.5.0 //variable
    pip       22.0.4
    setuptools 62.1.0
    sqlparse  0.4.2
    wheel     0.37.1
```

- instalar django-restframework, markdown y django-filter

Listing 43: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip install djangorestframework
pip install markdown
pip install django-filter
       //resultado:
       Package   Version
       ---------- -------
       asgiref   3.5.2
       Django    4.0.5 //Variable
       django-filter 23.2 //Variable
       djangorestframework 3.14.0 //Variable
       Markdown 3.4.3 //Variable
       pillow    9.5.0 //variable
       pip       22.0.4
       setuptools 62.1.0
       sqlparse  0.4.2
       wheel     0.37.1
```

- Registrando del restframework

Listing 44: Desarrollo en settings.py

```
INSTALLED_APPS = [
    ...
    'rest_framework',
 ]

 REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.DjangoModelPermissionsOrAnonReadOnly',
    ]
 }
```

- Creaciom de las urls, registrandolos en el restframework

Listing 45: Desarrollo en la principal urls.py

```
from rest_framework import routers, serializers, viewsets
  from django.conf import settings
```

```
#importamos los modelos
from Decoraciones.models import Tipo,Carrito,catalogo,Pedido

class TipoSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Tipo
        fields = ['Tip']

# ViewSets ...
class TipoViewSet(viewsets.ModelViewSet):
    queryset = Tipo.objects.all()
    serializer_class = TipoSerializer

class CarritoSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Carrito
        fields = ['nombre','precio','rutaI']

# ViewSets ...
class CarritoViewSet(viewsets.ModelViewSet):
    queryset = Carrito.objects.all()
    serializer_class = CarritoSerializer

class CatalogoSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = catalogo
        fields = ['nombre','precio','imagen','tipo']

# ViewSets define the view behavior.
class CatalogoViewSet(viewsets.ModelViewSet):
    queryset = catalogo.objects.all()
    serializer_class = CatalogoSerializer

class PedidoSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Pedido
        fields = ['NombreC','telefono','direccion','fecha']

# ViewSets define the view behavior.
class PedidoViewSet(viewsets.ModelViewSet):
    queryset = Pedido.objects.all()
    serializer_class = PedidoSerializer

router = routers.DefaultRouter()
router.register(r'Tipo', TipoViewSet)
router.register(r'Carrito', CarritoViewSet)
router.register(r'Catalogo', CatalogoViewSet)
router.register(r'Pedido', PedidoViewSet)

urlpatterns = [
    path('rest/', include(router.urls)),
    path('api-auth/',include('rest_framework.urls',
        namespace='rest_framework')),
    ...
] ...
```

- Cada serializador convierte los objetos del modelo en formato JSON y viceversa, y cada conjunto de vistas maneja las solicitudes HTTP para las operaciones CRUD utilizando los métodos HTTP correspondientes (POST, GET, PUT, DELETE). Por ejemplo, el conjunto de vistas "TipoViewSet"maneja las solicitudes HTTP para el modelo "Tipoütilizando los métodos HTTP POST, GET, PUT y DELETE.
- El **enrutador** de Django REST Framework se utiliza para **generar automáticamente las URL** para las vistas del conjunto, y se agregan a las URL de la aplicación principal utilizando la función ïnclude". También se agregan otras URL para la autenticación y la administración del sitio.
- **ACLARACION:** La clase producto no puede mostrase, debido a es una plantilla de las base de datos de otros models como carrito, y por ello no es una tabla en la base de datos en si.

Listing 46: Desarrollo en la principal urls.py

```
from django.db import models
   from django.contrib.auth.models import AbstractBaseUser,BaseUserManager
   # Create your models here.


   class UsuarioManager(BaseUserManager):
       def create_user(self, email, userName, nombres, apellidos, password=None):
           if not email:
               raise ValueError('El usuario requiere un email!')

           usuario = self.model(
               email=self.normalize_email(email),
               userName=userName, # Usa 'userName' en lugar de 'username'
               nombres=nombres,
               apellidos=apellidos,
           )

           usuario.set_password(password)
           usuario.save()
           return usuario

       def create_superuser(self, email, userName, nombres, apellidos, password):
           usuario = self.create_user(
               email,
               userName=userName,
               nombres=nombres,
               apellidos=apellidos,
               password=password,
           )
           usuario.usuario_admin = True
           usuario.save()
           return usuario


   class Usuario (AbstractBaseUser):
       userName = models.CharField('nombre de Usuario', unique= True, max_length=100)
       email = models.EmailField('Correo Electronico',unique=True, max_length=25)
       nombres = models.CharField('Nombres', max_length=50,blank=True,null=True)
       apellidos = models.CharField('Apellidos', max_length=250,blank=True,null=True)
       perfil = models.ImageField("Imagen",null=True, blank=True, upload_to="img/")
```

```
    usuario_activo = models.BooleanField(default = True)
    usuario_admin =models.BooleanField(default = False)
    objeto = UsuarioManager()

    USERNAME_FIELD = 'userName'
    REQUIRED_FIELDS = ['email','nombres','apellidos']

    def __str__(self):
        return f'{self.nombres},{self.apellidos}'


    def has_perm(self,perm,obj = None):
        return True

    def has_module_perms(self, app_label):
        return True


    @property
    def is_staff(self):
        return self.usuario_admin
```

- Este código define un modelo personalizado de usuario llamado 'Usuario' utilizando el framework Django. El modelo 'Usuario' hereda de 'AbstractBaseUser', lo que permite personalizar el sistema de autenticación y autorización en Django. A continuación, explicaré las partes importantes del código:
- from django.db import models': Importa las clases y funciones necesarias para definir modelos de base de datos en Django.
- from django.contrib.auth.models import AbstractBaseUser, BaseUserManager': Importa las clases necesarias para personalizar el modelo de usuario utilizando el sistema de autenticación de Django.
- UsuarioManager:
- Esta clase es un administrador personalizado que proporciona métodos para la creación de usuarios y superusuarios.
- createuser: Método para crear un usuario normal con un correo electrónico, nombre de usuario, nombres, apellidos y una contraseña opcional.
- createsuperuser: Método para crear un superusuario con los mismos campos que un usuario normal, pero con la diferencia de que se establece la propiedad 'usuarioadmin' en 'True'.
- Usuario (modelo de usuario personalizado):
- userName: Campo para el nombre de usuario, se define como un campo único ('unique=True') y tiene una longitud máxima de 100 caracteres.
- email: Campo para el correo electrónico del usuario, se define como un campo único ('unique=True') y tiene una longitud máxima de 25 caracteres.
- nombres: Campo para el nombre(s) del usuario, tiene una longitud máxima de 50 caracteres y puede ser opcional ('blank=True, null=True').

- apellidos: Campo para el apellido(s) del usuario, tiene una longitud máxima de 250 caracteres y puede ser opcional (‘blank=True, null=True‘).
- perfil: Campo para la imagen de perfil del usuario, es una imagen que se puede cargar en el directorio ïmg/ƶ es opcional (‘null=True, blank=True‘).
- usuarioactivo: Campo booleano que indica si el usuario está activo (‘True‘) o inactivo (‘False‘) de forma predeterminada se establece en ‘True‘.
- usuarioadmin: Campo booleano que indica si el usuario es un superusuario (‘True‘) o no (‘False‘) de forma predeterminada se establece en ‘False‘.
- objeto: Un objeto del administrador personalizado ‘UsuarioManager‘.
- USERNAMEFIELD: Especifica el campo que se utilizará como nombre de usuario para la autenticación (en este caso, ‘userName‘).
- REQUIREDFIELDS: Lista de campos requeridos además del ‘USERNAMEFIELD‘ al crear un usuario desde la línea de comandos o el formulario de administración.
- str(self): Método que devuelve una representación de cadena del usuario, en este caso, muestra los nombres y apellidos del usuario.
- hasperm(self, perm, obj=None)‘: Método que devuelve ‘True‘, lo que indica que el usuario tiene permiso para realizar cualquier acción.
- ‘hasmoduleperms(self, applabel)‘: Método que devuelve ‘True‘, lo que indica que el usuario tiene permisos en cualquier aplicación del proyecto.
- @property‘
- ‘isstaff‘: Un decorador ‘@property‘ que crea una propiedad virtual ‘isstaff‘. Si ‘usuarioadmin‘ es ‘True‘, esto indica que el usuario es miembro del personal/administrador.

Listing 47: Desarrollo en la principal urls.py

```
from typing import Any
  from django.contrib.auth.forms import AuthenticationForm
  from django.contrib.auth.forms import UserCreationForm
  from Usuario.models import Usuario
  from django import forms

  class FormularioLogin(AuthenticationForm):
      def __init__(self,*args, **kwargs):
          super(FormularioLogin, self).__init__(*args, **kwargs)
          self.fields['username'].widget.attrs['class'] = 'form-control'
          self.fields['username'].widget.attrs['placeholder'] = 'Nombre de Usuario'
          self.fields['password'].widget.attrs['class'] = 'form-control'
          self.fields['password'].widget.attrs['placeholder'] = 'Contrasena'

  class FormularioUser(forms.ModelForm):
      password1 = forms.CharField(label = "Contrasena", widget = forms.PasswordInput(
          attrs = {
                'class' : 'form-control',
                'placeholder' : 'Ingrese su contrasena ...' ,
                'id' : 'password1',
                 'required' : 'required',
```

```
            }
        ))
        password2 = forms.CharField(label = 'Contrasena de Confirmacion', widget =
            forms.PasswordInput(
            attrs = {
                  'class' : 'form-control',
                  'placeholder' : 'Ingrese nuevamente su contrasena ...' ,
                  'id' : 'password2',
                   'required' : 'required',
            }
        ))

        class Meta:
            model = Usuario
            fields = ('email','userName','nombres','apellidos')
            widgets = {
                'email': forms.EmailInput(
                    attrs={
                    'class': 'form-control',
                    'placeholder': 'Correo Electronico',
                    }
                ),
                'nombres': forms.TextInput(
                    attrs={
                    'class': 'form-control',
                    'placeholder': 'Ingrese su NOMBRE',
                    }
                ),
                'apellidos': forms.TextInput(
                    attrs={
                    'class': 'form-control',
                    'placeholder': 'Ingrese sus APELLIDOS',
                    }
                ),
                'userName': forms.TextInput(
                    attrs={
                    'class': 'form-control',
                    'placeholder': 'Ingrese su Nombre de Usuario',
                    }
                ),
            }

        def clean_password2(self):
            password1 = self.cleaned_data.get('password1')
            password2 = self.cleaned_data.get('password2')
            if(password1 and password2 and password1 != password2):
                raise forms.ValidationError('las contrasenas no coinciden')
            return password2
        def save(self, commit = True):
            user = super().save(commit = False)
            user.set_password(self.cleaned_data['password1'])
            if commit:
                user.save()
            return user
```

- Este código define dos clases de formulario personalizadas para autenticación y creación de usua-

rios en Django. A continuación, describiré cada una de ellas:

- 'FormularioLogin' (Formulario de inicio de sesión):
- Hereda de 'AuthenticationForm', que es un formulario proporcionado por Django para la autenticación de usuarios.
- En el método 'init', se personalizan los campos del formulario 'username' (nombre de usuario) y 'password' (contraseña) al agregar clases CSS y atributos de marcador de posición (placeholders) para dar estilo a los campos del formulario.
- El objetivo principal de este formulario es proporcionar una apariencia personalizada para el formulario de inicio de sesión, agregando estilos y atributos para mejorar la experiencia del usuario.
- 'FormularioUser' (Formulario de creación de usuarios):
- Hereda de 'forms.ModelForm', que es un formulario basado en modelos en Django que se utiliza para interactuar con modelos de base de datos existentes.
- Se definen dos campos adicionales 'password1' y 'password2' para la contraseña y su confirmación. Se utilizan widgets personalizados para definir las entradas de contraseña, con clases CSS y atributos de marcador de posición para mejorar la apariencia del formulario.
- La clase 'Meta' se utiliza para definir los campos del modelo 'Usuario' que se mostrarán en el formulario y los widgets personalizados para cada campo.
- El método 'cleanpassword2' se utiliza para verificar si las contraseñas ingresadas ('password1' y 'password2') coinciden. Si no coinciden, se lanza una excepción de formulario con un mensaje de error.
- El método 'save' se personaliza para guardar el usuario creado con la contraseña proporcionada. La contraseña se almacena utilizando el método 'setpassword', que cifra la contraseña antes de guardarla en la base de datos.

Listing 48: Desarrollo en la principal urls.py

```
from xhtml2pdf import pisa
   from django.template.loader import render_to_string
   from django.http import HttpResponseRedirect,HttpResponse
   from django.shortcuts import render, redirect
   from django.urls import reverse_lazy
   from django.db.models import Sum
   from Usuario.forms import FormularioUser,FormularioLogin
   from Usuario.models import Usuario
   from Decoraciones.models import catalogo,Carrito,Pedido
   from django.shortcuts import get_object_or_404
   from django.views.generic import CreateView
   from django.views.generic.edit import FormView
   from django.utils.decorators import method_decorator
   from django.views.decorators.cache import never_cache
   from django.views.decorators.csrf import csrf_protect
   from django.contrib.auth import login,logout
   from django.contrib.auth.decorators import login_required
   from django.db.models import F, ExpressionWrapper, FloatField, Sum

   class Login(FormView):
```

```
            template_name = 'login.html'
            form_class = FormularioLogin
            success_url = reverse_lazy('Decoraciones:Inicio')

            @method_decorator (csrf_protect)
            @method_decorator (never_cache)
            def dispatch(self, request, *args, **kwargs):
                if request.user.is_authenticated:
                    return HttpResponseRedirect(self.get_success_url())
                else:
                    return super(Login,self).dispatch(request, *args, **kwargs)

            def form_valid(self, form):
                login(self.request,form.get_user())
                return super(Login,self).form_valid(form)

        def logoutUsuario(request):
            logout(request)
            return HttpResponseRedirect('/accounts/login/')

        def deshacer_cambio(request, history_id):
            objeto_historial = get_object_or_404(catalogo.historial, history_id=history_id)
            producto = objeto_historial.history_object
            print(objeto_historial.history_object)
            if objeto_historial.get_history_type_display() == "Deleted":
                # Crear un objeto en catalogo para deshacer el cambio de tipo "Deleted"
                nuevo_objeto_catalogo = catalogo.objects.create(
                    nombre=producto.nombre,
                    descripcion=producto.descripcion,
                    precio=producto.precio,
                    Estado=producto.Estado,
                    imagen=producto.imagen,
                    tipo=producto.tipo
                )
                # Eliminar el objeto historial
                print("Restaurando cambio de tipo 'Deleted'")
            elif objeto_historial.get_history_type_display() == "Changed":
                # Restaurar el cambio de tipo "Changed"
                objeto_actual = catalogo.objects.get(id=producto.id)
                objeto_actual.nombre = producto.nombre
                objeto_actual.descripcion = producto.descripcion
                objeto_actual.precio = producto.precio
                objeto_actual.Estado = producto.Estado
                print(producto.precio)
                objeto_actual.imagen.Imagen = producto.imagen.Imagen
                objeto_actual.tipo = producto.tipo
                objeto_actual.save()
                # Eliminar el objeto historial
                print("Restaurando cambio de tipo 'Changed'")
            else:
                objeto_guardado = catalogo.objects.get(id=producto.id)
                objeto_guardado.delete()
            objeto_historial.delete()
            return redirect('Decoraciones:Historial')
```

```
# Create your views here.

#Lista de productos.

List_prod=Carrito.objects.all()

class regist_U(CreateView):
    model = Usuario
    form_class = FormularioUser
    template_name = 'registro.html'
    success_url = reverse_lazy('login')

    def post(self,request,*args,**kwargs):
        form = self.form_class(request.POST)
        if form.is_valid():
            nuevo_usuario = Usuario(
                email = form.cleaned_data.get('email'),
                userName = form.cleaned_data.get('userName'),
                nombres = form.cleaned_data.get('nombres'),
                apellidos = form.cleaned_data.get('apellidos'),
            )
            nuevo_usuario.set_password(form.cleaned_data.get('password1'))
            nuevo_usuario.save()
            return redirect('Decoraciones:Inicio')
        else:
            return render(request,self.template_name,{'form':form})
def Proforma(request):

    if request.method == "POST":
        contrato = Pedido(request.POST)
        if contrato.is_valid():
            contrato.save()
            contrato.carrito = Carrito.objects.all()
    else:
        contrato=Pedido()
    return render(request, 'Proforma.html',{"Ped":contrato,"productos":List_prod})

from django.contrib.auth.decorators import login_required

@login_required
def agregar(request):
    if request.method == "POST":
        nombreC = request.POST.get('nombre')
        descripcionC = request.POST.get('descripcion')
        precioC = request.POST.get('precio')
        urlC = request.POST.get('imagen_url')
        cantidad = request.POST.get('cantidad')
        comprador = request.user
        carrito = Carrito(nombre=nombreC, descripcion=descripcionC,
            precio=precioC, adquirido=True, cantidad=cantidad, rutaI=urlC,
            comprador=comprador)
        carrito.save()

    return redirect("Decoraciones:Inicio")

def borrar(request, numero):
```

```
        articulo = Carrito.objects.get(id=numero)
        articulo.delete()
        return redirect("usuario:bolsa")

    from django.db.models import F, Sum

    def lista(request):
        listaArticulo = Carrito.objects.all()
        total = listaArticulo.aggregate(total=Sum(F('precio') *
            F('cantidad')))['total'] or 0
        return render(request, 'lista.html', {"productos": listaArticulo, "total":
            total})


    @login_required
    def download_pdf(request):
        usuario = request.user.userName
        productos = Carrito.objects.filter(comprador=request.user)
        total = sum(producto.precio * producto.cantidad for producto in productos)

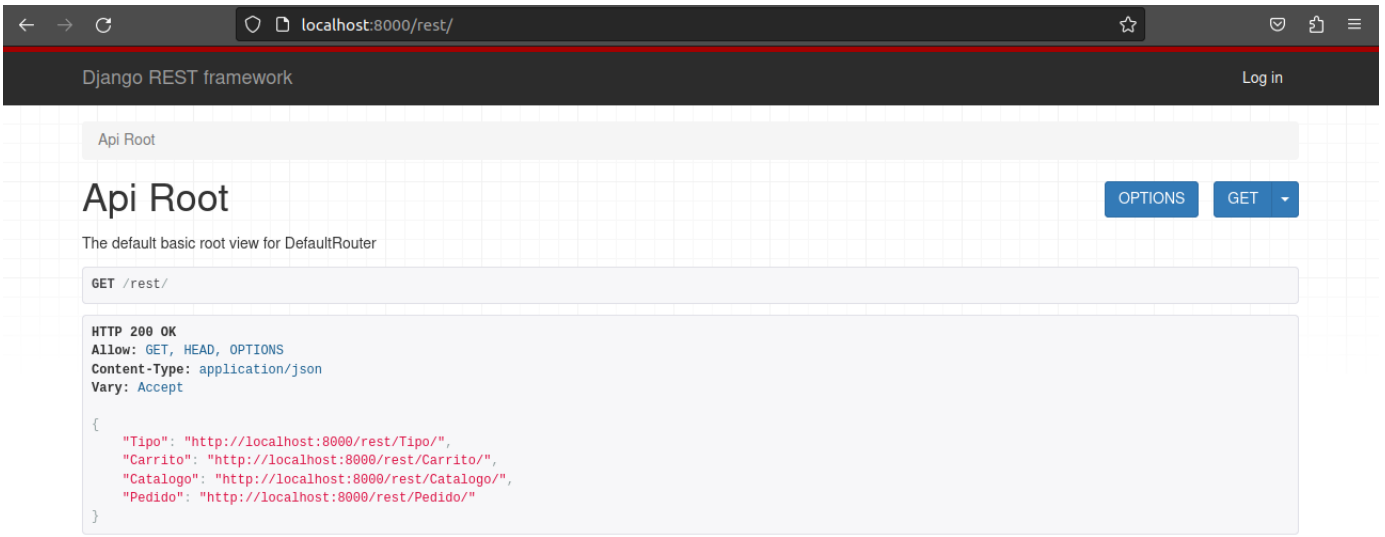
        # Renderizar el contenido del PDF usando el archivo de plantilla HTML
        html_content = render_to_string('pdf.html', {'usuario': usuario, 'productos':
            productos, 'total': total})

        # Convertir el contenido HTML a PDF
        response = HttpResponse(content_type='application/pdf')
        response['Content-Disposition'] = 'attachment; filename="reporte.pdf"'
        pisa.CreatePDF(html_content, dest=response)

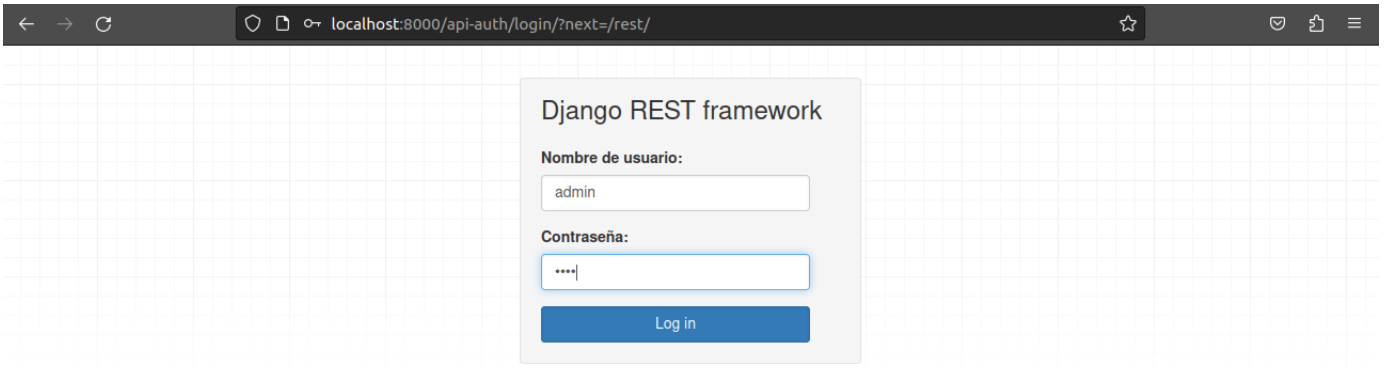
        return response
```

- downloadpdf:
- Esta función de vista permite generar un informe en formato PDF con los productos del carrito de compras del usuario.
- Obtiene el nombre de usuario y los productos del carrito (Carrito.objects.filter(comprador=request.user)).
- Calcula el total de la compra sumando el precio de cada producto multiplicado por su cantidad.
- Luego, renderiza el contenido del informe en una plantilla HTML (pdf.html) utilizando render-tostring.
- Convierte el contenido HTML en un PDF utilizando pisa.CreatePDF.
- Finalmente, devuelve el PDF generado para su descarga por el usuario.

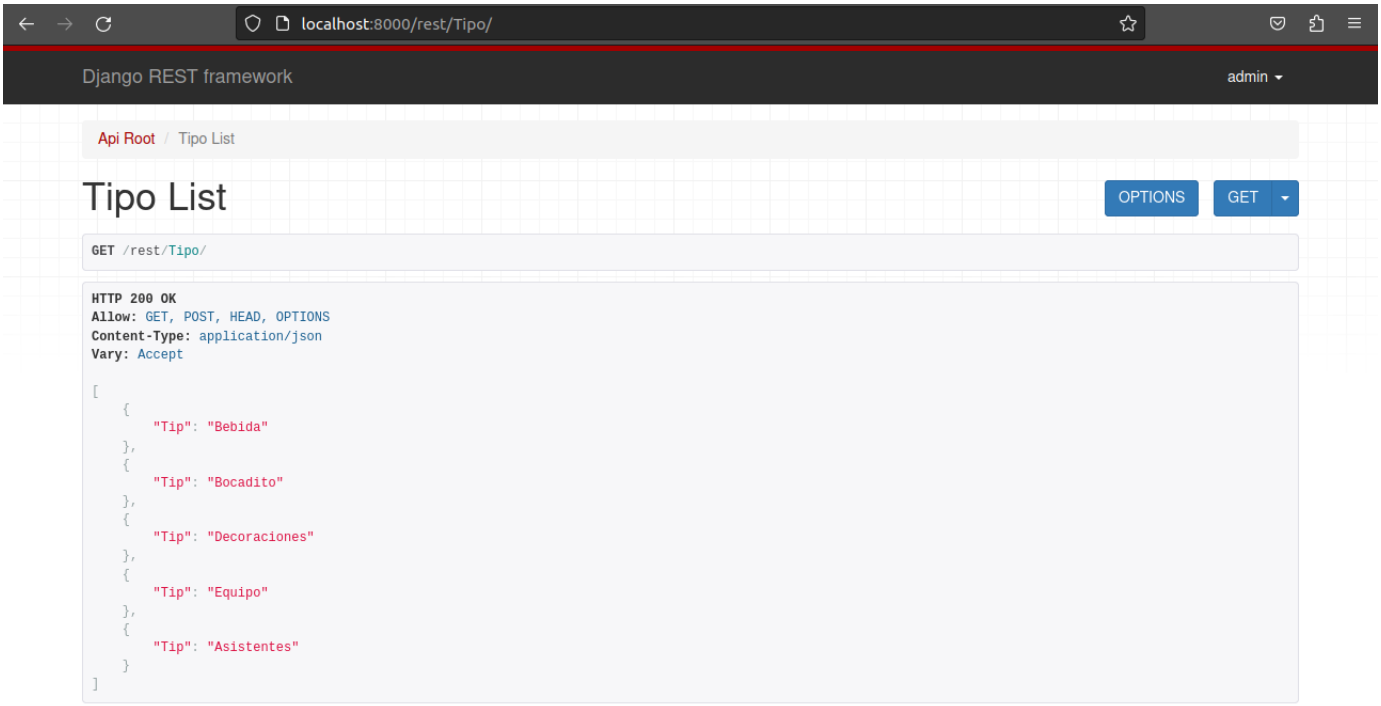
4.4. Ejecucion



- En este caso se observa la pagina del framework



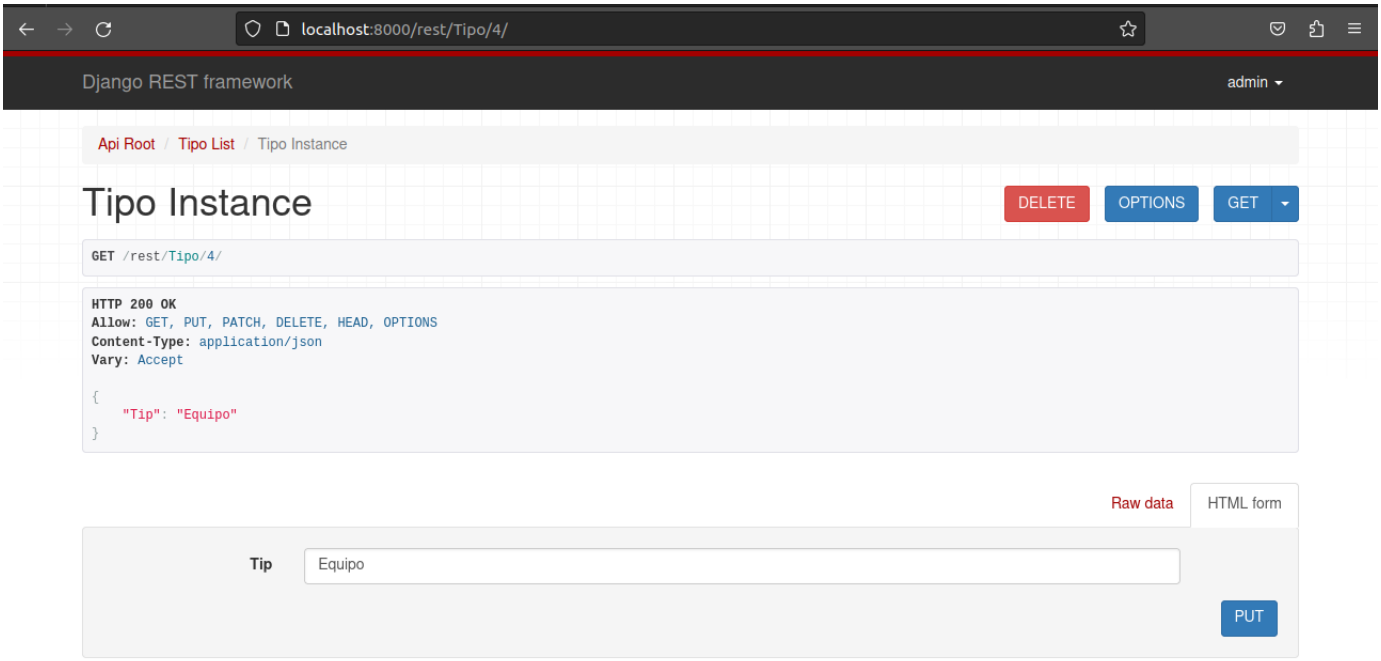
- Vista del login



- Vista del model Tipo

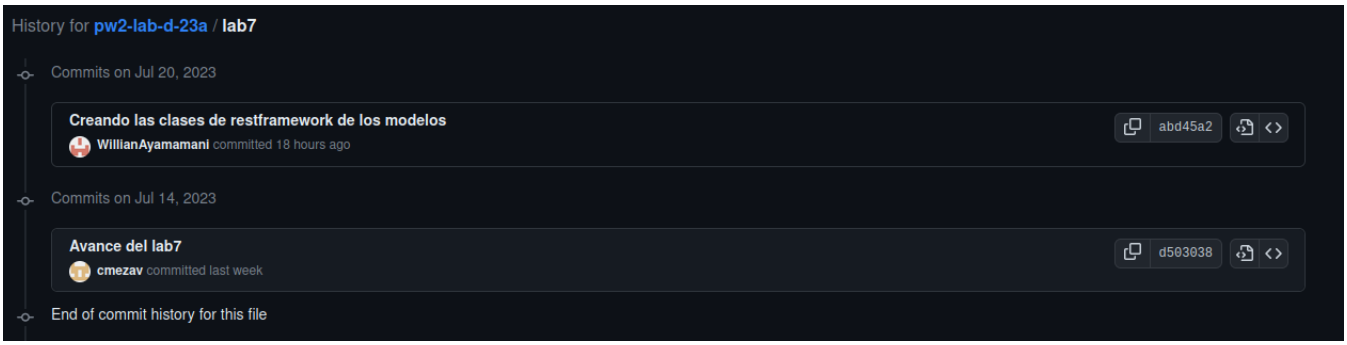


- Vista del PUT



- Vista del Delete, Put, Get; de un objeto.

4.5. Commit-GITUHUB



4.6. USUARIO

Listing 49: Credenciales

```
USUARIO:
    name:Lucho
    password:1234
 ADMIN:
    name:admin
    password:1234
```

4.7. Estructura de laboratorio 07

- El contenido que se entrega en este laboratorio es el siguiente:

---

5. Rúbricas

5.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

5.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	X	2	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	1	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	1	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	1	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	X	1	
Total		20		12	

6. Referencias

- https://github.com/WillianAyamamani/pw2-lab-d-23a/tree/main/lab7